

Empirical Analysis for “Predictive conditional alphas” (Distaso et. al 2026)

Paper authors: Valentina Corradi, Walter Distaso,
Marcelo Fernandes, and Asger Lunde
Research assistant: Ricardo Semião e Castro

16 Jul 2026

Table of contents

1	Empirical Analysis for “Predictive conditional alphas” (Distaso et. al 2026)	2
1.1	Repository organization	2
2	Data processing: factors	3
2.1	Data and methodology	3
2.2	Code setup	4
2.3	Importing and dates handling	4
2.4	Log-returns and 5-minute aggregation	7
2.5	Trading days	9
3	Data processing: stocks	11
3.1	Data and methodology	11
3.2	Code setup	12
3.3	Initial data cleaning	13
3.4	Dates analysis	16
3.5	Aggregation of 1-minute log returns	22
3.6	Appendix	25
4	Factor PCA	29
4.1	Methodology	29
4.2	Code setup	32
4.3	Code implementation	33
4.4	Calculating the PCA	36
4.5	Visualizing results	38
4.6	Appendix	41
5	Realized betas and risk-adjusted returns	43
6	Conditional alphas	44
7	References	45

1 Empirical Analysis for “Predictive conditional alphas” (Distaso et al. 2026)

This project contains the empirical analysis for the paper “Predictive conditional alphas” by Distaso et al. (2026). The relevant version of the paper can be found [bibliography/distaso2026_JASA.pdf](#).

The general steps of the empirical analysis are:

- Download and process factor and stock data.
- Extract high-frequency principal components from the factors.
- Estimate realized betas from the stock returns and the principal components.
- Estimate conditional alphas from the risk-adjusted returns.

The detailed methodology for each step is described in the associated report chapter or `.ipynb` notebook in `src/`.

This project has a repository and a PDF report associated. Both are explained in this text. Some links point to files in the repository and won’t work in the PDF report.

The project is run in python 3.14 and DuckDB. The `uv` config files allow for running the code with the correct package versions. To generate the report, run `src/quarto_render.ps1` at the project root. Or similar commands in Linux or MacOS.

1.1 Repository organization

The repository is organized as follows:

- `data`: not committed due to size.
 - `data/stocks_raw` and `stocks_1min_returns.parquet`: raw stocks from the TAQ database plus intermediate transformations, and the stocks log-returns at the 1-minute frequency.
 - `data/factors_1min_returns_raw.csv`, `data/factors_1min_returns.csv` and `data/factors_5min_returns.csv`: Factor returns at the 1-minute and 5-minute frequencies. The raw data is available at www.sakethaleti.com/data.
 - `data/pca_1min.csv` and `data/pca_5min.csv`: Principal components of the factors at the 1-minute and 5-minute frequencies.
- `src`: Most files here are sections of the report.
 - `src/data_factors.ipynb` `src/data_stocks.ipynb`: Data checking and processing.
 - `src/pca.ipynb`: High-frequency PCA of the factors estimation.
 - `src/betas.ipynb`: Realized betas and risk-adjusted returns estimation.
 - `src/alphas.ipynb`: Conditional alphas estimation.
 - `src/parameters.py`, `src/utls.py`: Utility scripts.
- `bibliography`: reference papers.
- `docs`: report of this empirical analysis.
- Configuration files:
 - Docs and repository: `__quarto.yml` `README.md` `.gitignore`
 - Python: `.python-version` `pyproject.toml` `uv.lock`

2 Data processing: factors

This notebook mines and processes the factor data needed. Read the [README.md](#)/Chapter 1 for context about the project.

2.1 Data and methodology

2.1.1 Raw data

Sources: Fama-French 6 factors from saketshaheti.com/data, or [this Google Drive link](#). Documentation can be found in the section 1 of the [online appendix](#).

Format: Already at the 1-minute level. One column for the minute, and one for each factor. The factors are in returns (not log).

Filters:

- Sample period (same as stocks):
 - Range: 2001-01-02 to 2017-12-29. Stocks that are listed or delisted during this period are included.
 - Hours: regular trading hours (09:30 - 16:00 ET).
 - Days: regular trading days, removing days with market-wide halts and diminished trading hours.

Qualities:

- The data should be order by `datetime`.
- There shouldn't be data for non-trading days, and there should be all 390 minutes of a day with data.
- The minutes should be left labelled as 9:30 - 9:30:00 -> 9:30:59, 9:31 - 9:31:00 -> 9:31:59, ..., 15:59 - 15:59:00 -> 15:59:59, and no additional 16:00 data point.

All are guaranteed by the source, and checked informally. The last is not, and we might need to transform it.

2.1.2 Methodology

The steps to process the data are as below.

- Download the data into `data/ff6_1min_returns_raw.csv`. Alternatively, could be read directly from Google Drive.
- Apply the filters above.
- Check if the minutes are labelled correctly, and transform if needed.
- Transform data into log-returns.
- Aggregate the data into left labeled 5-minute blocks (09:30, 09:35, ..., 15:55), by summing the returns of the label minute and the 4 minutes before it. Save it into `data/ff6_5min_returns.csv`.

2.2 Code setup

Codes for debugging:

```
%reset -f
import importlib
```

Core modules:

```
import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import polars as pl
import numpy as np

import src.utils as ut
import src.graphs as plot
from src.parameters import PARAMETERS as PARS
importlib.reload(ut)
importlib.reload(plot)
```

2.3 Importing and dates handling

Importing and formatting dates as datetime. A first look at the data is below. The names of the factors are:

- `ff__mkt`: market factor (MKT)
- `ff__smb`: size factor (SMB)
- `ff__hml`: value factor (HML)
- `ff__rmw`: profitability factor (RMW)
- `ff__cma`: investment factor (CMA)
- `ff__umd`: momentum factor (UMD)

```
data_raw = (
    pl.read_csv("data/factors_1min_returns_raw.csv")
    .with_columns(
        pl.col("datetime")
        .str.strptime(pl.Datetime, "%Y-%m-%d %H:%M:%S")
    )
)
cols_fct = ["ff__mkt", "ff__smb", "ff__hml", "ff__rmw", "ff__cma", "ff__umd"]

print(f>Data shape: {data_raw.shape}")
print("First rows:")
data_raw.head(3)
```

Data shape: (2454420, 7)

First rows:

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
1996-01-02 09:30:00	-0.002509	0.001133	-0.000592	-0.000541	-0.000922	-0.003535
1996-01-02 09:31:00	-0.000061	0.000184	0.000053	0.00002	-0.000021	0.000204
1996-01-02 09:32:00	-0.000123	0.000121	-0.000045	0.000123	-0.000098	-0.000083

Checking sorting:

```
if not data_raw["datetime"].is_sorted():
    raise ValueError("Data is not sorted by datetime.")
```

Filtering by date limits:

```
data_filt = data_raw.filter(
    (pl.col("datetime").dt.date() >= PARS.date_start.item())
    & (pl.col("datetime").dt.date() <= PARS.date_end.item())
)
```

All trading days from the NYSE calendar must be present, and all days in the data must be trading days:

```
dates_trading_all_exist = np.isin(
    PARS.trading_days,
    data_filt["datetime"].dt.date()
)
if np.any(~ dates_trading_all_exist):
    raise ValueError(f"There are {np.sum(dates_trading_all_exist)} missing days.")

dates_all_is_trading = np.isin(
    data_filt["datetime"].dt.date(),
    PARS.trading_days
)
if np.any(~ dates_all_is_trading):
    raise ValueError(f"There are {np.sum(dates_all_is_trading)} non-trading days.")
```

Checking non-full days (less than 391 minutes of data), and data where stocks have systematically few observations (see [src/data_stocks.ipynb](#) / Chapter 3). All are special days given by the NYSE's calendar. Will be removed.

```
dates_non_full = PARS.trading_days[
    ~ np.isin(PARS.trading_days, PARS.trading_days_final)
]
print(f"Non-full trading days:" + ut.format_days_byyear(dates_non_full))

data_filt = data_filt.filter(
    np.isin(data_filt["datetime"].dt.date(), PARS.trading_days_final)
)
```

Non-full trading days:

- 2001: 06-08, 07-03, 11-23, 12-24,
- 2002: 07-05, 09-11, 11-29, 12-24,
- 2003: 07-03, 08-15, 11-28, 12-24, 12-26,
- 2004: 11-26,
- 2005: 11-25, 12-23,
- 2006: 07-03, 11-24,
- 2007: 07-03, 11-23, 12-24,
- 2008: 07-03, 11-28, 12-24, 12-26,
- 2009: 11-27, 12-24,
- 2010: 11-26, 12-27,
- 2011: 08-15, 11-25, 12-23, 12-27,
- 2012: 07-03, 11-12, 11-23, 12-24,
- 2013: 07-03, 08-22, 11-29, 12-24,
- 2014: 07-03, 11-28, 12-24, 12-26,
- 2015: 10-12, 11-27, 12-24,
- 2016: 11-25, 12-23,
- 2017: 07-03, 11-24,

While the final factor data will be only for the defined window, we need warm up data for the PCA procedure of [src/pca.ipynb](#) Chapter 4. We need 858 observations of 1-minute data and 384 of 5-minute. We will add the raw data's start back in, do the transformations and aggregations, then keep only the needed window.

For this warm up data, we don't need matching with stocks nor full days, so those checks are not done.

```
dates_add = {1: 858, 5: 384}

data = pl.concat([
    data_raw.filter(data_raw["datetime"].dt.date() < PARS.date_start.item()),
    data_filt
])
```

Now lets alter the minute labels and remove overnight effects.

From Aleti (2022) online appendix: *"I only consider trades (i) between 09:30:00 and 16:00:00 [...]. I then collect low-frequency prices from CRSP to merge with the high-frequency prices from TAQ. [...] Using the adjusted close-to-close return along with the prices for each day, it is simple to infer the adjusted overnight (close-to-open) return. I can then assign this adjusted return to the 09:30:00 observation for each day. Additionally, I adjust the 16:00:00 return using the delisting return."*

Thus, the 9:30 observation is the overnight return and must be removed. The rest of the observations are right-labelled – 9:31 - 9:30:00 < x < 9:31:00, ... – with the last instant 16:00 being adjusted with the delisting return. Thus, they must be transformed into left-labels.

```
data = (
    data
    .filter(pl.col("datetime").dt.time() != pl.time(9, 30, 0))
    .with_columns(
        pl.col("datetime") - pl.duration(minutes = 1)
    )
)
```

2.4 Log-returns and 5-minute aggregation

Data is in returns, and indeed all values are in the range $[-1, 1]$. We can transform it into log-returns by applying the map $r \rightarrow \log(1 + r)$.

```
data_1min = data.with_columns(pl.col(cols_fct).log1p())

data_5min = (
    data_1min.group_by_dynamic("datetime", every = "5m")
    .agg([pl.col(col).sum() for col in cols_fct])
)
```

Then, remove the unneeded observations:

```
dates_start_idx = (
    np.flatnonzero(data_1min["datetime"].dt.date() >= PARS.date_start.item())[0]
)

data_1min = data_1min.slice(dates_start_idx - dates_add[1], None)

if (data_1min["datetime"].dt.date() < PARS.date_start.item()).sum() != dates_add[1]:
    raise ValueError("Wrong warm up size for 1-minute.")

dates_start_idx = (
    np.flatnonzero(data_5min["datetime"].dt.date() >= PARS.date_start.item())[0]
)

data_5min = data_5min.slice(dates_start_idx - dates_add[5], None)

if (data_5min["datetime"].dt.date() < PARS.date_start.item()).sum() != dates_add[5]:
    raise ValueError("Wrong warm up size for 1-minute.")
```

Having a look and saving the 1-minute data:

```
data_1min.write_csv("data/factors_1min_returns.csv", datetime_format = "%Y-%m-%d %H:%M")
data_1min
```

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
2000-12-27 14:42:00	-0.000078	0.000042	-0.000115	0.000057	0.000192	0.000248
2000-12-27 14:43:00	-0.000435	0.000376	0.000072	-0.000109	0.000194	-0.0003
2000-12-27 14:44:00	-0.000253	0.000221	0.000066	0.000196	-0.000238	-0.000078
2000-12-27 14:45:00	0.000304	-0.000367	-0.000153	0.00029	-0.000158	0.000021
2000-12-27 14:46:00	0.000057	-0.000287	-0.000133	0.000002	-0.0002	0.000121
...	...	...	...	...	...	...
2017-12-29 15:55:00	-0.00003	0.00007	-0.000064	0.000003	-0.00009	0.000055
2017-12-29 15:56:00	-0.000033	0.000039	-0.00004	0.000023	-0.000023	0.000103
2017-12-29 15:57:00	-0.000192	-0.000075	-0.000107	0.000056	0.000017	0.000075
2017-12-29 15:58:00	5.8310e-7	-0.000083	-0.000132	-0.000041	0.000018	-0.000061

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
2017-12-29 15:59:00	-0.001403	0.000706	-0.000426	0.000124	-0.000084	-0.00008

Having a look and saving the 1-minute data:

```
data_5min.write_csv("data/factors_5min_returns.csv", datetime_format = "%Y-%m-%d %H:%M")
data_5min
```

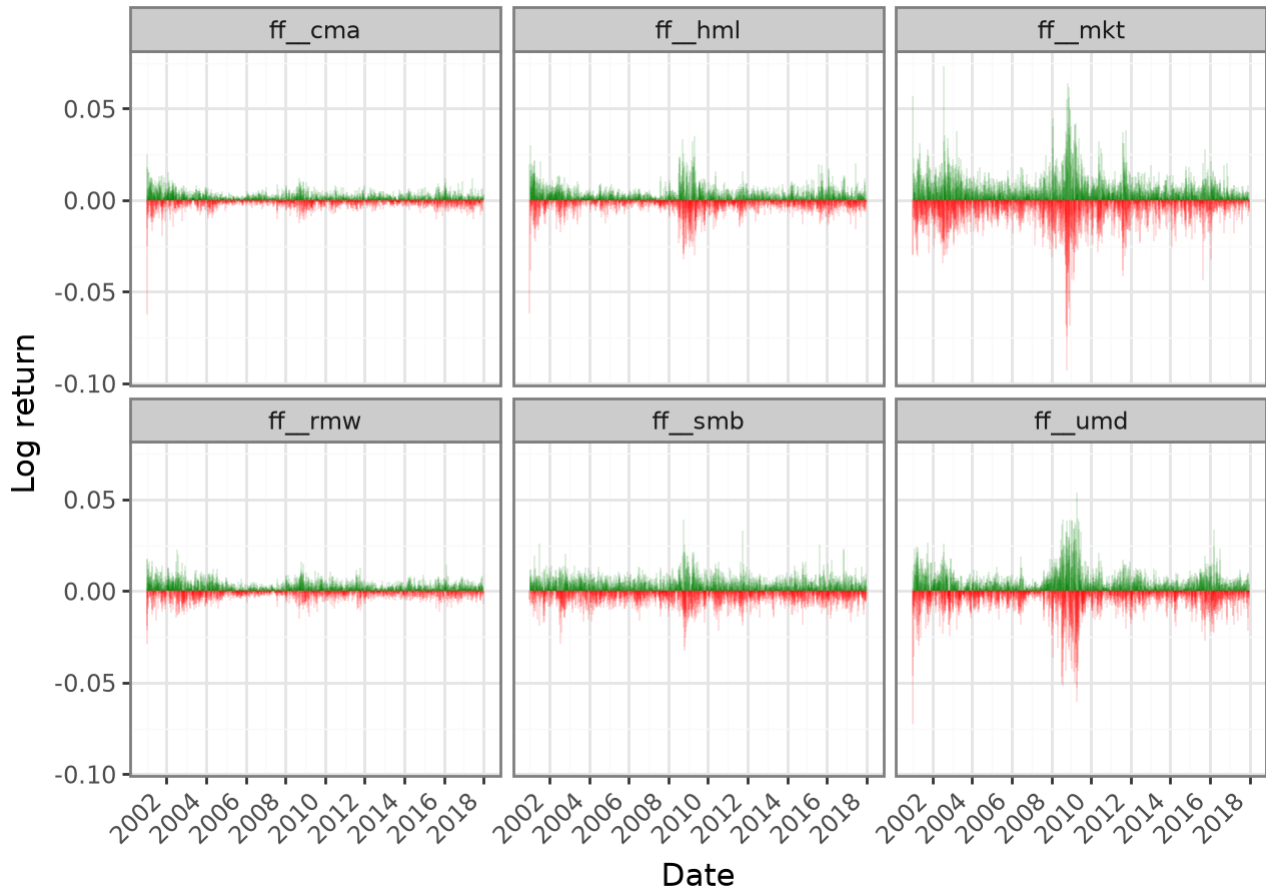
datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
2000-12-22 10:00:00	0.001716	0.000084	-0.00156	-0.001235	-0.001339	-0.000846
2000-12-22 10:05:00	-0.000355	0.001536	0.000538	-0.000224	0.00043	0.001022
2000-12-22 10:10:00	-0.001602	0.001063	0.001256	-0.000528	0.000825	0.000486
2000-12-22 10:15:00	0.000046	-0.000266	0.000518	-0.000772	-0.000284	-0.000367
2000-12-22 10:20:00	0.001853	-0.000933	-0.000773	-0.000039	-0.000956	0.000094
...	...	...	...	...	...	...
2017-12-29 15:35:00	-0.000758	0.000106	-0.000049	0.000123	-0.000138	0.000232
2017-12-29 15:40:00	-0.00028	0.000043	-0.000138	-0.000112	0.000141	0.000026
2017-12-29 15:45:00	0.000254	-0.000358	0.000196	-0.000303	0.000087	-0.000143
2017-12-29 15:50:00	-0.001415	-0.00086	0.000294	-0.000049	0.000542	-0.000387
2017-12-29 15:55:00	-0.001659	0.000658	-0.000769	0.000165	-0.000162	0.000091

2.4.1 Visualizing raw data and aggregation

The factors are in log returns, all centered around zero, with varying variance. The aggregations into 5-min blocks were a very simple transformation and won't be plotted here.

```
ut.plot.factors(data_1min, cols_fct)
```

Minute log returns of the Fama-French 6 factors



2.5 Trading days

Here I recall the approach used to define the full trading days considered in this project:

1. The trading days calendar for NYSE is obtained from the `exchange_calendars` and `pandas_market_calendars` python modules, checked against eachother. This is used to validate the raw stock and factor data.
2. Dates where there were less than 390 observations per minute in the factor data or simultaneously across all stocks are deemed as non-full trading days, and removed from the final list of trading days.

At the end, the following non-weekdays are removed from the final list of trading days:

```
dates_week_all = np.arange(PARS.date_start, PARS.date_end, dtype = 'datetime64[D]')
dates_week_all = dates_week_all[np.is_busday(dates_week_all)]
dates_week_rmv = dates_week_all[~ np.isin(dates_week_all, PARS.trading_days_final)]

print("Weekdays removed:" + ut.format_days_byyear(dates_week_rmv))
```

- 2001: 01-01, 01-15, 02-19, 04-13, 05-28, 06-08, 07-03, 07-04, 09-03, 09-11, 09-12, 09-13, 09-14, 11-22, 11-23, 12-24, 12-25,
- 2002: 01-01, 01-21, 02-18, 03-29, 05-27, 07-04, 07-05, 09-02, 09-11, 11-28, 11-29, 12-24, 12-25,

- 2003: 01-01, 01-20, 02-17, 04-18, 05-26, 07-03, 07-04, 08-15, 09-01, 11-27, 11-28
12-24, 12-25, 12-26,
- 2004: 01-01, 01-19, 02-16, 04-09, 05-31, 06-11, 07-05, 09-06, 11-25, 11-26, 12-24,
- 2005: 01-17, 02-21, 03-25, 05-30, 07-04, 09-05, 11-24, 11-25, 12-23, 12-26,
- 2006: 01-02, 01-16, 02-20, 04-14, 05-29, 07-03, 07-04, 09-04, 11-23, 11-24, 12-25,
- 2007: 01-01, 01-02, 01-15, 02-19, 04-06, 05-28, 07-03, 07-04, 09-03, 11-22, 11-23
12-24, 12-25,
- 2008: 01-01, 01-21, 02-18, 03-21, 05-26, 07-03, 07-04, 09-01, 11-27, 11-28, 12-24
12-25, 12-26,
- 2009: 01-01, 01-19, 02-16, 04-10, 05-25, 07-03, 09-07, 11-26, 11-27, 12-24, 12-25,
- 2010: 01-01, 01-18, 02-15, 04-02, 05-31, 07-05, 09-06, 11-25, 11-26, 12-24, 12-27,
- 2011: 01-17, 02-21, 04-22, 05-30, 07-04, 08-15, 09-05, 11-24, 11-25, 12-23, 12-26
12-27,
- 2012: 01-02, 01-16, 02-20, 04-06, 05-28, 07-03, 07-04, 09-03, 10-29, 10-30, 11-12
11-22, 11-23, 12-24, 12-25,
- 2013: 01-01, 01-21, 02-18, 03-29, 05-27, 07-03, 07-04, 08-22, 09-02, 11-28, 11-29
12-24, 12-25,
- 2014: 01-01, 01-20, 02-17, 04-18, 05-26, 07-03, 07-04, 09-01, 11-27, 11-28, 12-24
12-25, 12-26,
- 2015: 01-01, 01-19, 02-16, 04-03, 05-25, 07-03, 09-07, 10-12, 11-26, 11-27, 12-24
12-25,
- 2016: 01-01, 01-18, 02-15, 03-25, 05-30, 07-04, 09-05, 11-24, 11-25, 12-23, 12-26,
- 2017: 01-02, 01-16, 02-20, 04-14, 05-29, 07-03, 07-04, 09-04, 11-23, 11-24, 12-25,

3 Data processing: stocks

This notebook mines and processes the stock data needed. Read the [README.md](#)/Chapter 1 file for context about the project.

3.1 Data and methodology

3.1.1 Raw data

Sources: Intraday tick data from the [NYSE TAQ database](#). The relevant documentation is [Daily_TAQ_Client_Spec_v4.3.pdf](#).

Format:

The data is can be easily stored as an SQLite3 database with the main table **trades** of the following schema:

```
CREATE TABLE trades (  
  permno    INTEGER NOT NULL, -- Permanent number of the stock  
  ts_ny     INTEGER NOT NULL, -- Timestamp in New York time (epoch seconds)  
  price     REAL    NOT NULL, -- Price of the trade  
  size      INTEGER NOT NULL, -- Size of the trade  
  ex        TEXT    NOT NULL, -- Exchange code  
  corr      TEXT    NOT NULL, -- Correction code  
  ticker    TEXT    NOT NULL, -- Stock ticker symbol  
  ambiguous INTEGER NOT NULL, -- Flag if the trade is ambiguous (0 or 1)  
  PRIMARY KEY (permno, ts_ny)  
) WITHOUT ROWID
```

Further documentation might be found looking for the relevant websites.

For the confection of this notebook, the data was given in a private Dropbox folder. The TODO file contains a script to download it into the `data/stocks_raw/trades.db` file.

Qualities:

The data is expected to have the qualities below. All are guaranteed by the data source.

- Within each **permno**, the data should be ordered by **ts_ny**. *Will be sorted.*
- There is one obs per minute, at the 00 seconds mark. Duplicates don't exist but missing minutes do. *Will be checked.*
- Data validity: **price** should never be negative or zero; **ex** and **corr** should have only the correct levels. *Anomalies are filtered out.*
- There should be data only for trading days. *Will be checked.*

3.1.2 Methodology

First, some initial filters are applied (Section 3.3) and the raw data is saved in an optimized format. Then, the data is transformed from a permno ~ second ~ price data into a permno ~ minute ~ log return data (Section 3.5).

The calculations are done via DuckDB SQL queries.

3.2 Code setup

Codes for debugging and running SQL inside notebooks:

```
%reset -f
import importlib

from IPython.core.magic import register_cell_magic
from pprint import pprint

import duckdb

queries_registered: dict[str, str] = {}

@register_cell_magic
def register_query(line: str, cell: str) -> None:
    query_name = line.strip()
    if not query_name:
        raise ValueError("You must provide a name for the query.")
    queries_registered[query_name] = cell
    return None
```

Setting up DuckDB infrastructure. The threads and memory limits are set following the memory constraints (16GB RAM) of the machine used to process the data.

```
%%register_query duckdb_infra
INSTALL sqlite;
LOAD sqlite;

-- DETACH db_trades;
ATTACH IF NOT EXISTS 'data/stocks_raw/trades.db' AS db_trades (TYPE SQLITE);

SET enable_progress_bar = true;
SET enable_progress_bar_print = true;

PRAGMA threads = 7;
PRAGMA memory_limit = '14GB';
```

```
duckdb.execute(queries_registered["duckdb_infra"])
```

Core modules:

```

import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import math

import polars as pl
import numpy as np

import src.utils as ut
import src.graphs as plot
from src.parameters import PARAMETERS as PARS
importlib.reload(ut)
importlib.reload(plot)

```

3.3 Initial data cleaning

An initial filter is applied before the processing logic, to remove unnecessary data.

The column types are converted to ones that occupy less space in memory. See the [DuckDB types](#) for reference.

The is then saved into a [.parquet](#) file, a columnar storage format that is more efficient for analytical queries and compression than the original SQLite3 database. The row group size is chosen as approximately the median of observations per `permno`.

Filters:

- Exchanges: NYSE, NYSE-Arca, NASDAQ, and AMEX (code shares 10 and 11 at the CRSP database) (`ex IN ('P', 'N', 'Q', 'A')`).
- Types of trades: regular trades (`corr = '00'`).
- Price should be positive (`price > 0`).
- Sample period(same as factors):
 - Range: 2001-01-02 to 2017-12-29. Stocks that are listed or delisted during this period are kept.
 - Hours: regular trading hours (09:30:00 - 15:59:59 ET).

Formatting and data checking:

- `permno`: is a 5-digit positive integer, and is stored as UINT32.
- `ts_ny`:
 - Is a seconds-since-1970-01-01 timestamp.
 - Our start date is 2001-01-02, which guarantees that the relevant observations are always positive.
 - There are no duplicates (`ts_ny > LAG(ts_ny)`).
 - There is only one observation per minute at the 00 mark `ts_ny % 60 != 0`, thus it is transformed into a minutes epoch.
 - Stored as UINT32, whose maximum date goes up to the year 10141.
- `price`: 14 integer digits and 6 decimal digits, stored as FLOAT8. `Decimal(9, 6)` could be used, but python's matrix libraries work better with floats.

The two checks are stored into a validation column, and the final result is partitioned by it. Rows are partitioned into the `data/stocks_raw` directory's sub-folders `valid=pass/`, `valid==fail_not_minute/`, and `valid==fail_unordered_or_dupe/` (not committed to git).

This will be implemented via the following query:

```
%%register_query raw_to_clean

-- ?1: start epoch seconds (PARS.date_start)
-- ?2: end epoch seconds (PARS.date_end)

COPY (
  SELECT
    CAST(permno AS UINT32) AS permno,
    CAST(ts_ny // 60 AS UINT32) AS ts_min_ny,
    CAST(price AS FLOAT8) AS price,
    CASE
      WHEN (COUNT(*) OVER (PARTITION BY permno, ts_min_ny)) > 1
        AS is_duplicate
      THEN 'fail_duplicate'
      ELSE 'pass'
    END AS valid
  FROM db_trades.trades
  WHERE
    ts_ny BETWEEN ?1 AND ?2
    AND corr = '00'
    AND ex IN ('P', 'N', 'Q', 'A')
    AND (ts_ny % 86400) BETWEEN 34200 AND 57599 -- 09:30:00 to 15:59:59
    AND price > 0
  ORDER BY permno, ts_min_ny -- Costly but helps with querying later
) TO 'data/stocks_raw' (
  FORMAT PARQUET,
  COMPRESSION 'ZSTD',
  ROW_GROUP_SIZE 555000, -- Median of observations per permno
  PARTITION_BY (valid)
);
```

Then, we check if there are no data in the `valid==fail_duplicate/` folders, and move the `valid=pass/` data into the `data/stocks_raw` folder:

```
if False: # Only execute once
    duckdb.execute(
        queries_registered["raw_to_clean"],
        [
            int(x.astype("datetime64[s]").astype(np.int64))
            for x in (PARS.date_start, PARS.date_end)
        ]
    )
    os.rename(
        "data/stocks_raw/valid=pass/data_0.parquet",
        "data/stocks_raw/trades_clean.parquet"
    )
```

```

if "valid==fail_duplicate/" in os.listdir("data/stocks_raw/"):
    raise ValueError(
        "There are invalid rows to be analyzed. Check the `data/stocks_raw/`'s"
        " subfolders"
    )

```

For utility, save the number of observations for each permno:

```

%%register_query counts_permno
COPY (
    SELECT
        permno,
        COUNT(*) as obs_count
    FROM 'data/stocks_raw/trades_clean.parquet'
    GROUP BY permno
    ORDER BY permno
) TO 'data/stocks_raw/counts_permno.csv' (HEADER, DELIMITER ',');

```

```

if False: # Only execute once
    duckdb.execute(query = queries_registered["counts_permno"])

data_counts_permno = pl.read_csv("data/stocks_raw/counts_permno.csv")

```

The number of observations (1-minute blocks) for each (permno, day):

```

%%register_query counts_permno_day_1min
COPY (
    SELECT
        permno,
        (ts_min_ny // (60 * 24)) as d,
        COUNT(*) as obs_count
    FROM 'data/stocks_raw/trades_clean.parquet'
    GROUP BY permno, d
) TO 'data/stocks_raw/counts_permno_day_1min.csv' (HEADER, DELIMITER ',');

```

```

if False: # Only execute once
    duckdb.execute(query = queries_registered["counts_permno_day_1min"])

```

And the number of 5-minute blocks with data, for each (permno, day):

```

%%register_query counts_permno_day_5min
COPY (
    SELECT
        permno,
        (ts_min_ny // (60 * 24)) as d,
        COUNT(DISTINCT ts_min_ny // 5) as obs_count
    FROM 'data/stocks_raw/trades_clean.parquet'
    GROUP BY permno, d
) TO 'data/stocks_raw/counts_permno_day_5min.csv' (HEADER, DELIMITER ',');

```

```
if False: # Only execute once
    duckdb.execute(query = queries_registered["counts_permno_day_5min"])
```

Saving the data, and filling missing days within the permno lifetime with `obs_count = 0`:

```
data_counts_min = {
    x: pl.read_csv(f"data/stocks_raw/counts_permno_day_{x}min.csv")
    for x in [1, 5]
}
```

3.4 Dates analysis

Now, we turn to analyzing the distribution of observations within a day for a stock. We want to remove stocks that have too many days with none or too few observations.

3.4.1 Trading days

First, we need to define what is a missing day, that is, what are the trading days. In [src/parameters.py](#), we calculate the full (9:30 → 16:00) trading days as the union of: days with full factor data, days as described by the `exchange_calendars` python package, and days described by the `pandas_market_calendars` package. But, there might be some days that were not full trading days for the stocks in our sample.

First, let's check if any of the stocks have data for non-trading days. It is expected that there are data for non-full trading days.

```
data_days = data_counts_min[1].with_columns(
    pl.from_epoch(pl.col("d"), time_unit = "d")
)

days_unique = np.unique(data_days["d"].to_numpy().astype("datetime64[D]"))

if not np.isin(days_unique, PARS.trading_days).all():
    raise ValueError("There are days in the data that are not trading days.")

if not np.isin(days_unique, PARS.trading_days_factors).all():
    print("There are days in the data that are not full trading days.")
```

There are days in the data that are not full trading days.

The river plot below shows, for each day, the quantiles of the number of minutes with data for each stock, divided by the total number of minutes in a trading day (390).

The blue dots show days where the factor data didn't have all minutes with data, which at first, is our truth source for non-full days. The gray dots show the days where the median is -0.1 points smaller than its 5-day rolling mean, that is, possible extraordinary trading days.

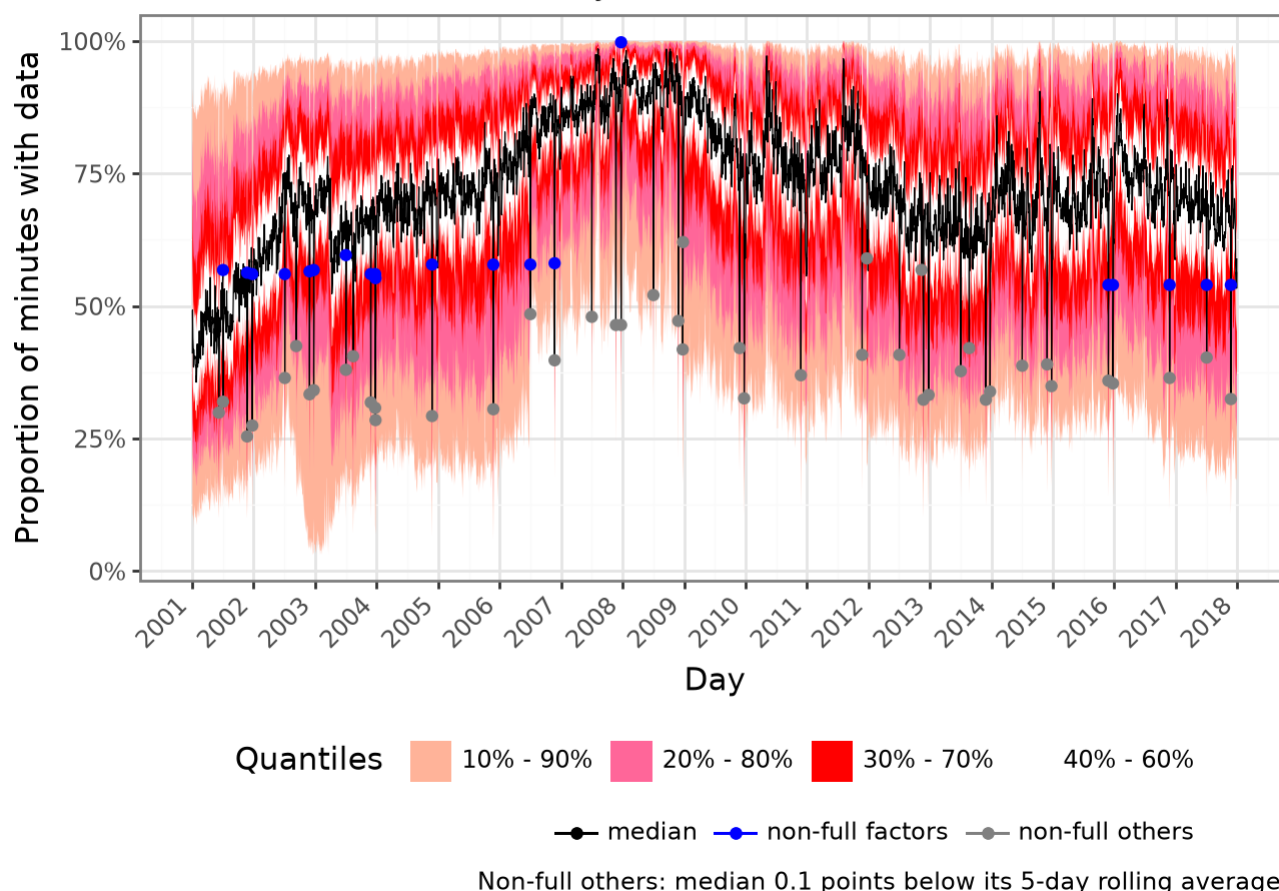
```

data_days_fill = (
    data_days
    .with_columns(pl.col("obs_count") / PARS.blocks_1min)
    .group_by("d")
    .agg(
        median = pl.median("obs_count"),
        **{
            f"q{q}": pl.quantile("obs_count", q / 10)
            for q in [4, 6, 3, 7, 2, 8, 1, 9]
        }
    )
    .sort("d")
)

ut.plot.day_fullness_river(data_days_fill, data_counts_permno.shape[0])

```

Proportion of minutes with data within each day
Quantiles calculated each day across all 2364 stocks



The average median is ≈ 0.72 . We can see many outlier dates with very low coverage, which are likely to be extraordinary trading days. Still, some of these days are not within the list of full trading days, as shown below. Individually checking the dates, they still seem to be extraordinary trading days, but that have data present for the factors. They will be included in the non-full trading days list.

```

days_low_obs = (
    data_days_fill

```

```

        .with_columns(median_rw = pl.col("median").rolling_mean(8))
        .filter(pl.col("median") - pl.col("median_rw") <= - 0.1)
        ["d"].to_numpy()
    )

print(f"Number of days with low observations: {len(days_low_obs)}")
print(ut.format_days_byyear(days_low_obs[
    np.isin(days_low_obs, PARS.trading_days_factors)
]))

```

Number of days with low observations: 52

```

- 2001: 06-08
- 2002: 09-11
- 2003: 08-15
- 2005: 12-23
- 2007: 07-03, 11-23
- 2008: 07-03, 11-28, 12-24, 12-26
- 2009: 11-27, 12-24
- 2010: 11-26, 12-27
- 2011: 08-15, 11-25, 12-23, 12-27
- 2012: 07-03, 11-12, 11-23, 12-24
- 2013: 07-03, 08-22, 11-29, 12-24
- 2014: 07-03, 11-28, 12-24, 12-26
- 2015: 10-12
- 2016: 12-23

```

3.4.2 Days with missing data

Now, on to checking which days have missing data. Missing days within the PERMNO's lifetime are filled with 0. For each 1-minute and 5-minute block, we plot:

1. The histogram of the number of observed blocks for each day, across all days of all stocks.
2. Boxplots of the proportion of days with more observed blocks than some threshold τ , across all stocks, for multiple thresholds.

```

data_filled_min = {
    x: ut.fill_missing_days(data_counts_min[x])
    for x in [1, 5]
}

```

3.4.3 1-Minute blocks

For $\tau = 78$, there are 1,292 stocks with at most 2.5% non-full days.

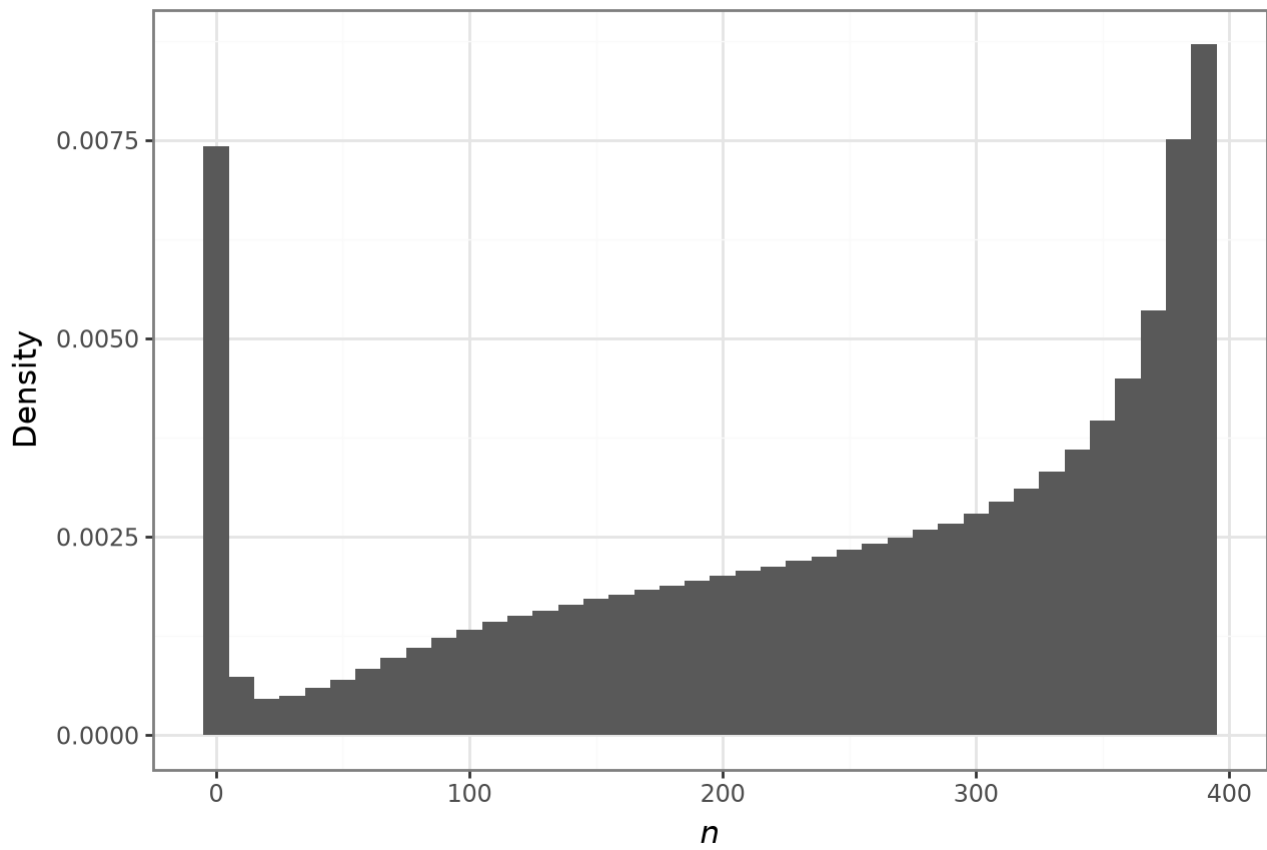
```

ut.plot.day_fullness_hist(data_filled_min[1])

```

Distribution of days' observation count (n)

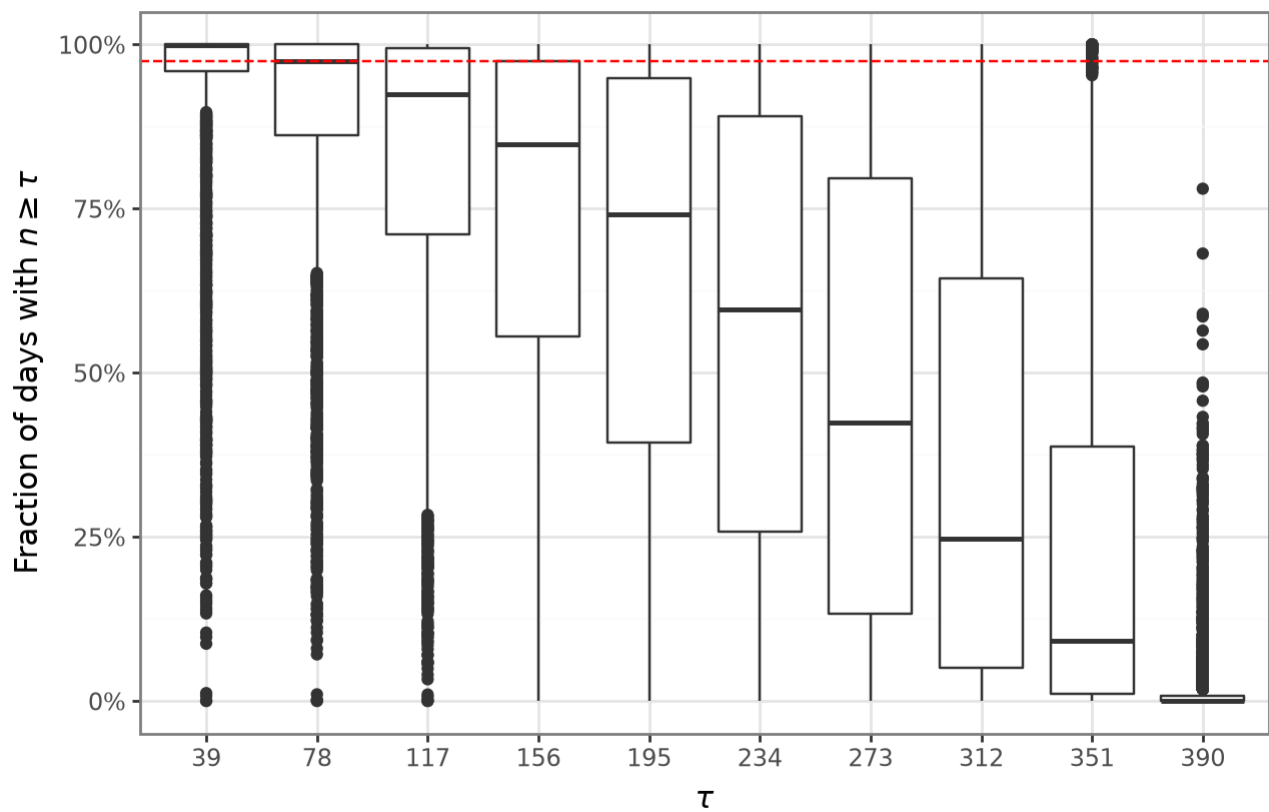
Across all 6,397,249 stocks-day pairs



```
ut.plot.day_fullness_box(data_filled_min[1], 1, data_counts_permno.shape[0])
```

Distribution of days' observation count (n)

Across all 2364 stocks



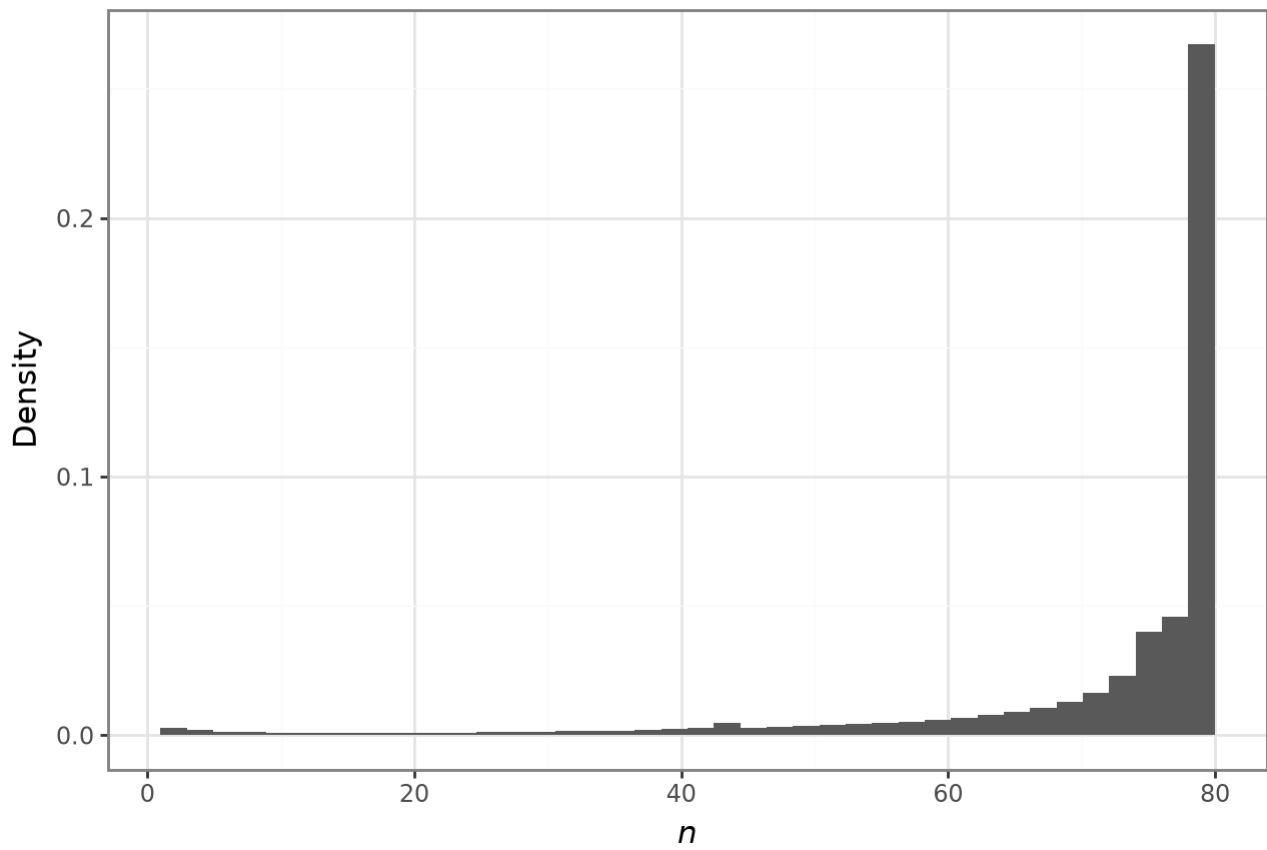
Red line: 97.5%.

3.4.4 5-minute blocks

```
ut.plot.day_fullness_hist(data_counts_min[5])
```

Distribution of days' observation count (n)

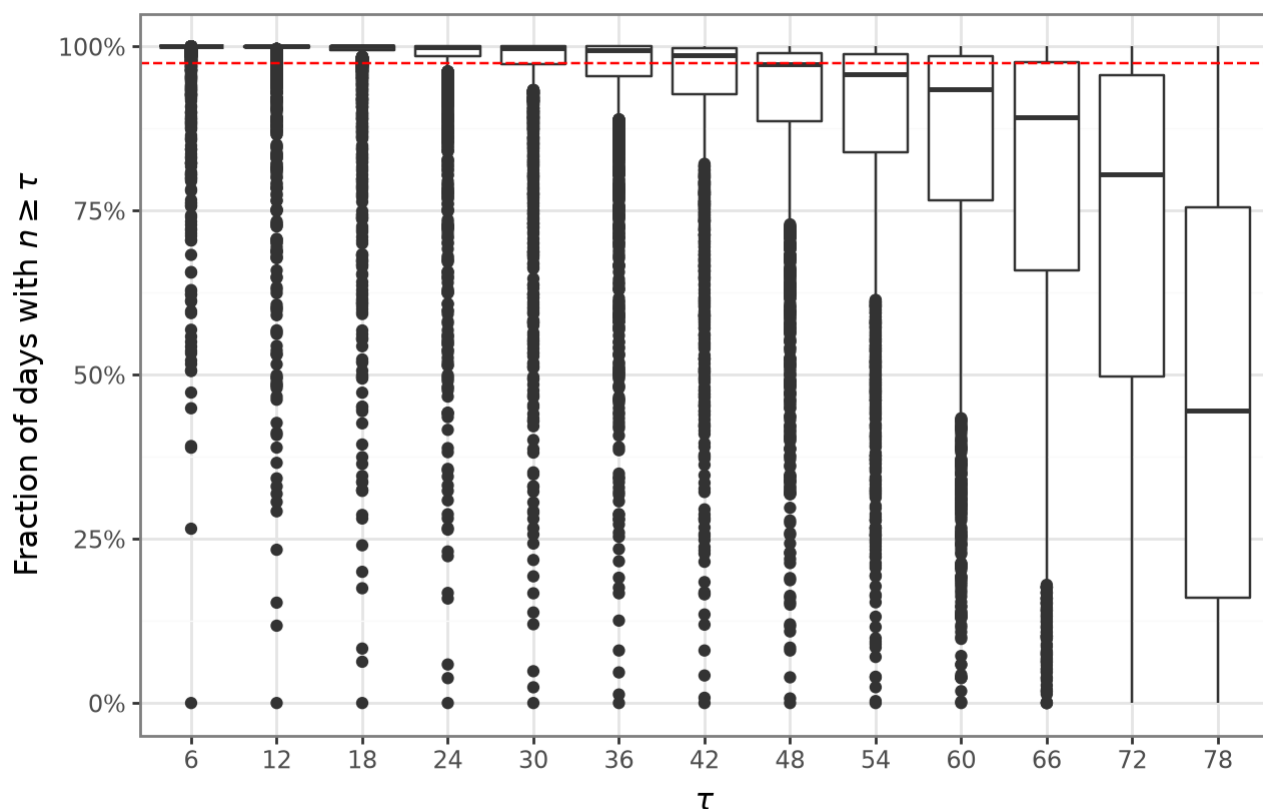
Across all 6,054,309 stocks-day pairs



```
ut.plot.day_fullness_box(data_counts_min[5], 5, data_counts_permno.shape[0])
```

Distribution of days' observation count (n)

Across all 2364 stocks



Red line: 97.5%.

3.5 Aggregation of 1-minute log returns

To aggregate the data, for each permno:

1. For each 1-minute block m : get the last seen price before the end of m 's block, p_m .
2. Minimal liquidity filter: at most 2.5% of the trading days (excluding pre-listing and post-delisting days) have less than 78 minutes with data.
3. Calculate $r_m^1 = \ln(p_t/p_{m-1})$.
4. Exclude the first return of the day to remove overnight effects.
5. Exclude non-full trading days..

- Save a [data/stocks_1min_returns.parquet](#) file with the columns `permno`, `ts_min_ny`, and `log_return`.

Some notes:

- As checked in the raw data, the observations are already at the minute level with no duplicates, so we don't need to actually do any aggregation.
- The "last seen price" is not the undisputed correct price to use for calculating returns. Other methods include the midpoint of the bid-ask spread and the volume-weighted average price.
- We could save the information of how many observations were within each minute block, but that information is not needed at the moment.

The transformation could be done with either DuckDB or Polars, both multithreaded. It also could be done in chunks if the data is too big. Polars seems to deal better with the latter, but the following DuckDB code is written to process the data in one go. See the appendix for alternatives.

```
%%register_query trade_returns_aggregation_chunked

-- ?1: list of permnos to filter
-- ?2: chunk file path

COPY (
  WITH
  trades_by_min AS (
    SELECT
      permno,
      ts_min_ny,
      price,
      (ts_min_ny // (60 * 24)) AS d
    FROM 'data/stocks_raw/trades_clean.parquet'
    WHERE
      permno BETWEEN ?1[0] AND ?1[-1]
  )
  SELECT
    permno,
    ts_min_ny,
    LN(price / LAG(price, 1) OVER w) AS log_return
  FROM trades_by_min
  WINDOW w AS (PARTITION BY permno, d ORDER BY ts_min_ny)
  QUALIFY
    LAG(price, 1) OVER w IS NOT NULL
)
TO ?2 (
  FORMAT PARQUET,
  COMPRESSION 'ZSTD',
  ROW_GROUP_SIZE 555000
);
```

```
duckdb.execute(queries_registered["trade_returns_aggregation_chunked"])
```

The data will be forward filled with 0 return on minutes with no trades, and will also be aggregated to the 5-minute level. But, storing these to disk is unneeded, and it is better to create the variations on-the-fly.

3.5.1 Visualizing raw data and aggregation

Lets visualize the work to have a first look at the data and check if everything is as expected.

First, lets sample some random stocks.

```
vis_permnos = data_counts_permno["permno"].sample(4).to_numpy()
```

We could visualize the original raw data, but very few operations were done on it. Still, one could run the following SQL and polars codes to generate a dataset to visualize:

```

%%register_query raw_permno_sample

-- ?1: list of integer permnos to filter

SELECT
    CAST(permno AS UINT32) AS permno,
    CAST(ts_ny // 60 AS UINT32) AS ts_min_ny,
    CAST(price AS FLOAT8) AS price,
FROM db_trades.trades
WHERE
    permno IN ?1

```

```

data_permno_raw = (
    duckdb.sql(queries_registered["raw_permno_sample"], params = [vis_permnos.tolist()])
    .pl()
    .with_columns(
        datetime = pl.from_epoch(pl.col("ts_min_ny") * 60, time_unit = "s")
    )
    .select(["permno", "datetime", "price"])
)

```

The code below plots the clean prices and the returns aggregation. Both are aggregated to the 30-day level, to make the relationship between the two clearer.

The relationship between the two looks as expected.

```

ut.plot.prices_returns(
    pl.scan_parquet("data/stocks_raw/trades_clean.parquet")
    .filter(pl.col("permno").is_in(vis_permnos))
    .collect(),
    pl.scan_parquet("data/stocks_1min_returns.parquet")
    .filter(pl.col("permno").is_in(vis_permnos))
    .collect()
)

```

Price and log returns across time

Each column is a randomly selected PERMNOs



3.6 Appendix

If the calculations above are too harsh for the available machine's memory, one can do it in chunks of `permno` values. The chunks are defined below, striving for a balance in the number of rows in each chunk. Each chunk has 7% of the rows, a number defined by the memory constraints (16GB RAM) of the machine used to process the data.

```
agg_chunk_fraction = 0.07
agg_chunk_cutoff = math.floor(
    int(data_counts_permno["obs_count"].sum()) * agg_chunk_fraction * 0.95
) # 0.95 avoids last chunk being too small
agg_total_chunks = math.ceil(1 / agg_chunk_fraction)

chunks = []
current_stock = 0

for i in range(agg_total_chunks):
    c = []
    c_rows = 0
    while c_rows < agg_chunk_cutoff and current_stock < len(data_counts_permno):
        c.append(data_counts_permno["permno"][current_stock])
        c_rows += data_counts_permno["obs_count"][current_stock]
        current_stock += 1
    if i == agg_total_chunks - 1:
```

```

        c.extend(data_counts_permno["permno"][current_stock:].to_list())
        chunks.append(c)

if sum(len(c) for c in chunks) != len(data_counts_permno):
    raise ValueError("Some stocks are missing from the chunks.")

print(f"Number of chunks: {len(chunks)}")
print("Number of stocks in each chunk: ", end = "")
pprint([len(c) for c in chunks], width = 50, compact = True)
print("Rows/1M per chunk (approx): ", end = "")
pprint([
    round(
        sum(data_counts_permno.filter(pl.col("permno").is_in(c))["obs_count"])
        / 1_000_000,
        1
    )
    for c in chunks
], width = 50, compact = True)

```

```

Number of chunks: 15
Number of stocks in each chunk: [168, 328, 103, 106, 111, 110, 106, 141, 142, 150,
166, 164, 163, 190, 216]
Rows/1M per chunk (approx): [107.0, 108.1, 107.7, 107.6, 107.1, 107.5, 107.0,
106.9, 107.3, 107.7, 107.9, 106.9, 107.1, 107.0,
103.3]

```

```

data_trades_lazy = pl.scan_parquet(
    "data/stocks_raw/trades_clean.parquet",
    parallel = "row_groups",
    low_memory = False
)

def lazy_pipeline_agg(chunk: list[int]) -> pl.LazyFrame:
    trading_days_extra = PARS.trading_days[
        ~ np.isin(PARS.trading_days, PARS.trading_days_final)
    ].astype("int32")

    pipeline = (
        data_trades_lazy
        .filter(pl.col("permno").is_between(chunk[0], chunk[-1]))
        #.sort(["permno", "ts_min_ny"]) # Already sorted
        .with_columns(
            d = (pl.col("ts_min_ny") // (60 * 24))
        )
        .with_columns(
            log_return = (
                pl.col("price") / (
                    pl.col("price")
                    .shift(1).over(["permno", "d"]) # Never cross days
                )
                # order_by = "ts_min_ny" is unneeded
            )
        )
    )

```

```

        ).log(),
    )
    .filter(
        pl.col("log_return").is_not_null() # Excludes first null return of day
        & (~ pl.col("ts_min_ny").is_in(trading_days_extra))
    )
    .select(["permno", "ts_min_ny", "log_return"])
)
return pipeline

```

For each chunk, the pipeline is executed and the results are saved in a parquet file such as [data/stocks_1min_returns/chunk_1.parquet](#):

```

if False: # Only execute once
    for i, chunk in enumerate(chunks[-1:]):
        print(f"Processing chunk {i+1}/{len(chunks)} with {len(chunk)} permnos")
        lazy_pipeline_agg(chunk).sink_parquet(
            f"data/stocks_1min_returns/chunk_{i}.parquet",
            compression = "zstd",
            engine = "streaming",
            row_group_size = 555_000
        )

```

An alternative to the code for Polars is the DuckDB code below.

```

%%register_query trade_returns_aggregation_chunked

-- ?1: list of permnos to filter
-- ?2: chunk file path

COPY (
    WITH
    trades_by_min AS (
        SELECT
            permno,
            ts_min_ny,
            price,
            (ts_min_ny // (60 * 24)) AS d
        FROM 'data/stocks_raw/trades_clean.parquet'
        WHERE
            permno BETWEEN ?1[0] AND ?1[-1]
    )
    SELECT
        permno,
        ts_min_ny,
        LN(price / LAG(price, 1) OVER w) AS log_return
    FROM trades_by_min
    WINDOW w AS (PARTITION BY permno, d ORDER BY ts_min_ny)
    QUALIFY
        LAG(price, 1) OVER w IS NOT NULL
)

```

```
TO ?2 (
    FORMAT PARQUET,
    COMPRESSION 'ZSTD',
    ROW_GROUP_SIZE 555000
);
```

```
if False: # Run polars instead
    for i, chunk in enumerate(chunks[-1:]):
        print(f"Processing chunk {i+1}/{len(chunks)} with {len(chunk)} permnos")
        duckdb.execute(
            queries_registered["trade_returns_aggregation_chunked"],
            [chunk, f"data/stocks_1min_returns/chunk_{i}.parquet"]
        )
```

4 Factor PCA

Now, we need to get the principal components of the fama-french 6 factors.

Standard PCA suffers from:

- **Ccurse of dimensionality:** estimating covariance matrices for large cross-sections requires long time horizons to avoid singularity.
- **Non stationarity:** the hypothesis of a constant covariance matrix across time can be too strong in some scenarios, this one included.
- **Non linearities:** PCA on a single covariance matrix, a linear measure of relationships, for the whole process, cannot capture non-linear relationships properly.

Here we apply Aït-Sahalia and Xiu (2015), that uses high frequency data to estimate spot covariance matrices, and the principal components from those. With it, we relax the stationarity assumptions needed and approximate non-linear relationships given Itô's Lemma.

The following math and commentary is taken largely as-is from the paper. First, I present the theory, then the implementation in Python, and finally, the results

4.1 Methodology

4.1.1 Dynamics of the variable

The d -dimensional (in our case $d = 6$) process X of the factors is a general Itô-semimartingale, defined on a filtered probability space:

$$X_t = X_0 + \int_0^t b_s ds + \int_0^t \sigma_s dW_s + J_t \quad (2)$$

- X_t is the d -dimensional log-price (or cumulative factor) process.
- b_s is the d -dimensional locally bounded drift vector.
- σ_s is the spot volatility matrix, linking to the spot covariance via $c_s = \sigma_s \sigma_s^T$.
- W_s is a d -dimensional standard Brownian motion.
- J_t is a pure jump process accounting for discrete, anomalous shocks. (More info on the paper).

4.1.2 Parameters of interest

The PCA goal is to find instantaneous linear combinations of X , orthogonal to each other, that maximize some measure of variation. In the paper's case, the continuous part of the quadratic variation.

Let $\mathcal{M}_d^+$ denote the set of $d \times d$ positive semi-definite symmetric matrices. For any $A \in \mathcal{M}_d^+$, we define:

- The d -dimensional vector of its eigenvalues sorted in non-increasing order: $\lambda(A) = (\lambda_1(A), \lambda_2(A), \dots, \lambda_d(A))^T$.
- The eigenvector corresponding to the g -th eigenvalue $\lambda_g(A)$ as $\gamma_g(A)$, which satisfies: $A\gamma_g = \lambda_g\gamma_g$ and $\gamma_g^T \gamma_g = 1$.

Our interest is calculating the $T \times d$ matrix of principal components, where T is the total number of (high-frequency) observations. This is related to the paper's derivation of the integrated/realized principal component of the g -th eigenvalue λ_g , $\int_0^t \gamma_{g,s-}^T dX_s$.

The paper focus on the estimation of the realized eigenvalues ($\int_0^t \lambda_s ds$) and eigenvectors too ($\int_0^t \gamma_{g,s} ds$), while deriving their asymptotic properties and bias corrections. I will focus on the PCs, but at Section 4.6 I summarize the assumptions for the asymptotic properties and the unused estimators.

4.1.3 Principal components estimator

4.1.3.1 Spot covariance matrix

The first step for all is estimating a spot covariance matrix and the eigenvalues associated with it. To capture non-stationary dynamics, the time window $[0, t]$ in increments of Δ_n is divided into non-overlapping blocks of length $k_n \Delta_n$. Within each small block, the continuous covariance c_s is assumed to be roughly constant. By dynamically estimating instantaneous matrices and summing them, the procedure continuously adapts to shifting correlations, capturing non-linearities without needing global parameter estimates.

At each $ik_n \Delta_n$, we estimate $c_{ik_n \Delta_n}$ using the block spot covariance estimator $\hat{c}_{ik_n \Delta_n}$ defined as:

$$\hat{c}_{ik_n \Delta_n} = \frac{1}{k_n \Delta_n} \sum_{j=1}^{k_n} (\Delta_{ik_n+j}^n X) (\Delta_{ik_n+j}^n X)^T \mathbf{1} \{ \|\Delta_{ik_n+j}^n\| \leq u_n \} \quad (5)$$

Where the return increment is: $\Delta_l^n X = X_{l\Delta_n} - X_{(l-1)\Delta_n}$, and u_n is the jump truncation threshold. The k and u parameters can be chosen with the shorthands proposed by the literature:

- **Block Size (k_n):** the divisor of $[t/\Delta_n]$ closer to $\theta \Delta_n^{-1/2} \sqrt{\log(d)}$, with $\theta = 0.5$. Other choices of θ and functions of d deliver similar results. It scales inversely with the sampling frequency (Δ_n) and grows slightly with dimensionality (d) to ensure stable covariance estimates without smoothing over true time-varying dynamics.
- **Jump Truncation ($u_{i,n}$):** Set asset-by-asset as $u_{i,n} = 3 \left(\int_0^t c_{ii,s} ds / t \right)^{0.5} \Delta_n^{0.47}$. Where the internal integral can be estimated with Bipower Variation. This snips off discrete jumps while preserving continuous diffusion.

The instantaneous eigenvalues $\hat{\lambda}_{ik_n \Delta_n} = \lambda(c_{ik_n \Delta_n})$ are obtained from the standard determinant equation:

$$\det(\hat{c}_i - \hat{\lambda} \mathbb{I}) = 0$$

4.1.3.2 Integrated principal component

The paper focuses on the integrated principal components, whose estimator is defined as:

$$\widehat{PC}_g = \sum_{i=1}^{[t/(k_n \Delta_n)]-1} \hat{\gamma}_{g,(i-1)k_n \Delta_n}^T (X_{(i+1)k_n \Delta_n} - X_{ik_n \Delta_n}) \quad (\text{Proposition 3})$$

Because the estimator below is a stochastic integral, the weight errors are uncorrelated with the forward returns. The estimation noise naturally cancels out, removing the need for a Hessian bias correction. PCs use the eigenvectors estimated from the immediately preceding block to project the

current block's returns, ensuring that our dynamic portfolio weights are strictly “adapted” (determined in-sample before trading) and free from microscopic look-ahead bias.

But, we want a $T \times d$ matrix of principal components, not just the integrated one. So, we basically don't want to run the summation, and store the intermediate results. The pseudo-code below outlines the algorithm.

4.1.4 Estimator algorithm

To estimate the $T \times d$ matrix of principal components, we must project each individual high-frequency return observation onto the previous block's eigenvector. The pseudocode belows outlines the procedure.

This code was written in matrix form, as it will be much more effective way to implement in Python. Still, it should be equivalent to the original paper's algorithm.

```

procedure pca_high_freq(
    X : Matrix;           // Shape: (T, d) | High-frequency return matrix
    k : Integer;         // Scalar       | Observations per block
    u : Matrix;           // Shape: (d,)   | Jump truncation thresholds
    evectors_prev : Matrix // Shape: (d, d) | Initial eigenvectors
);

    // 1. Parameter Setup & Initialization
    T = number of rows in X
    d = number of cols in X
    Delta = 1 / T
    N = Floor(T / k) // Total number of full blocks

    // Initialize empty arrays to store outputs:
    PC = empty Matrix of size T x d
    E_vals = empty Matrix of size N x d
    E_vecs = empty Matrix of size N x d x d

    // 2. Process Data Block by Block
    for i = 0 to N - 1 do
    begin
        // Define the current block
        block_start = i * k
        block_end = (i + 1) * k
        X_block = X[block_start to block_end]

        // Calculate PCs for the current block using the previous eigenvectors:
        PC[block_start to block_end] = X_block * evectors_prev

        // Truncate X with boolean mask and calculate the spot covariance matrix:
        X_trunc := X_block * (Abs(X_block) <= u)
        cov_spot = (Transpose(X_trunc) * X_trunc) / (k * Delta)

        // Eigen decomposition:
        evalues, evectors = eigen decomposition of cov_spot
        evalues, evectors = sort in descending order of evalues
    
```

```

        // Store results and update eigenvectors for the next block:
        E_vals[i, :] = evalues
        E_vecs[i, :, :] = evectors
        evectors_prev = evectors
    end;

    // 3. Handling the last block if (T % k != 0)
    block_start = N * k
    if block_start < T then
    begin
        X_block = X[block_start to T]
        PC[block_start to T] = X_block * evectors_prev
    end;

    return PC, E_vals, E_vecs
end;

```

4.1.5 Specifics of our setup

I will implement the above algorithm for the Fama-French 6 factors. One important thing to add is that we are treating the timeline with gaps on non-business days, non-trading hours, and removed 9:30 return. This is treated as a continuous timeline for the PCA. While this is standard, it is still important to note.

4.2 Code setup

For debugging:

```

%reset -f
import importlib

```

Core modules:

```

import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import polars as pl
import numpy as np

from numpy.typing import NDArray

import src.utils as ut
import src.graphs as plot
from src.parameters import PARAMETERS as PARS
importlib.reload(ut)
importlib.reload(plot)

```

4.3 Code implementation

Here, I implement the algorithm described above in Python.

First, we implement the rules of thumb used in the literature to set the parameters k and u :

```
def default_k(d: int, T: int, Delta: float) -> int:
    theta = 0.5

    k_target = int(np.round(theta * (Delta**-0.5) * np.sqrt(np.log(d))))
    divisors = np.array([i for i in range(1, T + 1) if T % i == 0])
    closest_idx = np.argmin(np.abs(divisors - k_target))

    return int(divisors[closest_idx])

def default_u(X: NDArray, T: int, Delta: float) -> NDArray:
    X_abs = np.abs(X)
    bpv = (np.pi / 2) * np.sum(X_abs[1:] * X_abs[:-1], axis = 0)
    u = 3 * np.sqrt(bpv / 1) * (Delta ** 0.47) # bpv/1:  $\sigma^2$  normalized to 1
    return u
```

Then, to start the PCA algorithm, we need to estimate the first block's eigenvectors. We can use the warm up data selected in [src/data_factors.ipynb](#) / Chapter 2 to achieve that goal. Else, the first block would be left with `np.nan`.

```
def pca_start_evecors(
    X: NDArray, k: int | None = None, u: NDArray | None = None,
    quiet: bool = True
) -> NDArray:
    ut.pca_high_freq_validate(X, k, u, None)

    T: int = X.shape[0]
    d: int = X.shape[1]
    Delta: float = 1 / T

    if k is None:
        k = default_k(d, T, Delta)
        if not quiet: print(f"Calculated `k` as {k}.")
    if u is None:
        u = default_u(X, T, Delta)
        if not quiet: print(f"Calculated `u` as {u}.")

    _, evecors = pca_truncated(X, k * Delta, u)

    return evecors
```

Finally, we implement the main algorithm. First, the simple PCA with truncation engine, for the stationary data within each block:

```
def pca_truncated(
    X: NDArray, # Shape: (k, d) a block of the original X
    n: int | float, # k * Delta
```

```

    u: NDArray # Shape: (d,)
):
    X_trunc = X * (np.abs(X) <= u) # x*0 for big values, else x*1
    cov_spot = (X_trunc.T @ X_trunc) / n

    evalues, evectors = np.linalg.eigh(cov_spot)
    evalues_order = np.argsort(evalues)[::-1]

    return evalues[evalues_order], evectors[:, evalues_order]

```

Then, the main loop, that iterates over the blocks, running the PCA, and storing results:

```

def pca_high_freq_core(
    N: int, k: int, T: int, Delta: float,
    u: NDArray, # Shape: (d,)
    X: NDArray, # Shape: (T, d)
    evectors_prev: NDArray, # Shape: (d, d)
    PC: NDArray, # Shape: (T, d)
    E_vals: NDArray, # Shape: (N, d)
    E_vecs: NDArray, # Shape: (N, d, d)
    quiet: bool = False
):
    for i in range(N): # Last block after loop
        if not quiet: print(f"- block {i + 1} of {N + 1}.")
        block_start, block_end = i * k, (i + 1) * k

        X_block = X[block_start:block_end, :]

        PC[block_start:block_end, :] = X_block @ evectors_prev

        evalues_prev, evectors_prev = pca_truncated(X_block, k * Delta, u)
        E_vals[i, :] = evalues_prev
        E_vecs[i, :, :] = evectors_prev

    block_start = N * k
    if block_start < T:
        if not quiet: print(f"- block {N + 1} of {N + 1}.")
        X_block = X[block_start:T, :]
        PC[block_start:T, :] = X_block @ evectors_prev

    return PC, E_vals, E_vecs

```

At last, the main orchestrator, that validates the inputs, sets the default parameters, and runs the main loop:

```

def pca_high_freq(
    X: NDArray, # Shape: (T, d)
    k: int | None = None,
    u: NDArray | None = None, # Shape: (d,)
    evectors_start: NDArray | None = None, # Shape: (d, d)
    quiet: bool = False

```

```

) -> dict[str, NDArray]:
    # Setup:
    ut.pca_high_freq_validate(X, k, u, evectors_start)
    X.flags.writeable = False

    T: int = X.shape[0] # Number of observations (rows)
    d: int = X.shape[1] # Number of variables (columns)
    Delta: float = 1 / T

    if k is None:
        k = default_k(d, T, Delta)
        if not quiet: print(f"Calculated `k` as {k}.")
    if u is None:
        u = default_u(X, T, Delta)
        if not quiet: print(f"Calculated `u` as {u}.")

    # Warm up:
    N: int = T // k # Number of blocks - 1
    if evectors_start is None:
        evectors_prev = np.full((d, d), np.nan)
    else:
        evectors_prev = evectors_start # Shape: (d, d)

    PC = np.full((T, d), np.nan) # Shape: (T, d)
    E_vals = np.full((N, d), np.nan) # Shape: (N, d)
    E_vecs = np.full((N, d, d), np.nan) # Shape: (N, d, d)

    # Main loop:
    if not quiet: print(f"Starting PCA with {N} blocks of size {k}.")
    PC, E_vals, E_vecs = pca_high_freq_core(
        N = N, k = k, T = T, Delta = Delta, u = u,
        X = X, evectors_prev = evectors_prev,
        PC = PC, E_vals = E_vals, E_vecs = E_vecs,
        quiet = quiet
    )

    return {"pc": PC, "evalues": E_vals, "evectors": E_vecs}

```

Some performance considerations:

- The `pca_high_freq_core()` must be run sequentially.
- Given the low dimensionality of the factor data, using GPU with e.g. `cupy` would add unnecessary overhead. Still the code could be translated basically as-is with `import cupy as np` instead.
- It could even be beneficial to lock multithreading of numpy's BLAS/LAPACK matrix engines.
- Finally, the code could be just-in-time compiled with `numba`, and the separation of `pca_high_freq_core()` helps that. One would only need to decorate it with `@numba.jit`.

The code is running super fast, such that these complications are not needed.

4.4 Calculating the PCA

4.4.1 Loading data

First, I load the data processed in [src/data_factors.ipynb](#) / Chapter 2.

```
blocks_size = {1: 858, 5: 384}

data_raw = {
    1: pl.read_csv("data/factors_1min_returns.csv"),
    5: pl.read_csv("data/factors_5min_returns.csv")
}

data_raw = {
    min: df.with_columns(
        pl.col("datetime").str.strptime(pl.Datetime, format = "%Y-%m-%d %H:%M")
    )
    for min, df in data_raw.items()
}

cols_fct = ["ff__mkt", "ff__smb", "ff__hml", "ff__rmw", "ff__cma", "ff__umd"]
```

Then, we separate the added wam-up data from the main part, from which we want the PCs:

```
data_warmup = {
    min: (
        df
        .filter(pl.col("datetime").dt.date() < PARS.date_start.item())
        [cols_fct]
        .to_numpy()
    )
    for min, df in data_raw.items()
}

data_main = {
    min: df.filter(pl.col("datetime").dt.date() >= PARS.date_start.item())
    for min, df in data_raw.items()
}
```

4.4.2 Running algorithm

First, let's inspect the default k and u parameters that will be generated. It only cuts the higher of quantiles of the data.

```
pca_start_evecors(data_main[1][cols_fct].to_numpy(), quiet = False)
u_default = [0.00182358, 0.0012769, 0.00082141, 0.00055697, 0.00055211, 0.00102862]

print("\nCalculated `u` and quantiles of the absolute factors:")
(
    pl.concat(
        [pl.DataFrame({col: val * 100 for col, val in zip(cols_fct, u_default)})]
```

```

    + [
        data_main[1].select(cols_fct).select(pl.all().abs().quantile(q) * 100)
        for q in [0.995, 0.99, 0.5]
    ]
)
.with_columns(
    stat = pl.Series(["u"] + [f"factor's Q{q}" for q in [0.995, 0.99, 0.5]])
)
.select(["stat"] + cols_fct)
)

```

Calculated `k` as 858.

Calculated `u` as [0.00182358 0.0012769 0.00082141 0.00055697 0.00055211 0.00102862].

Calculated `u` and quantiles of the absolute factors:

stat	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
str	f64	f64	f64	f64	f64	f64
"u"	0.182358	0.12769	0.082141	0.055697	0.055211	0.102862
"factor's Q0.995"	0.191411	0.127203	0.08511	0.052444	0.052304	0.110336
"factor's Q0.99"	0.149494	0.101947	0.067479	0.042692	0.042699	0.086969
"factor's Q0.5"	0.013916	0.010854	0.00696	0.005668	0.005523	0.007894

Then, we run the results for each minute-block-size:

```

pca_1min = pca_high_freq(
    X = data_main[1][cols_fct].to_numpy(),
    evectors_start = pca_start_evectors(data_warmup[1]),
    quiet = True
)

```

```

pca_5min = pca_high_freq(
    X = data_main[5][cols_fct].to_numpy(),
    evectors_start = pca_start_evectors(data_warmup[5]),
    quiet = True
)

```

Saving results:

```

n_pcs = pca_1min["pc"].shape[1]
cols_pc = {f"column_{i}": f"pc_{i + 1}" for i in range(n_pcs)}

```

```

pcas = {1: pca_1min, 5: pca_5min}

```

```

data_pcs = {
    k: (
        pl.DataFrame(res["pc"])
        .rename(cols_pc)
    )
}

```

```

        .with_columns(
            datetime = data_main[k]["datetime"],
            block = pl.row_index() // blocks_size[k]
        )
        .select(["datetime", "block"] + list(cols_pc.values()))
    )
    for k, res in pcas.items()
}

```

```

data_pcs[1].write_csv("data/pca_5min.csv", datetime_format = "%Y-%m-%d %H:%M")
data_pcs[5].write_csv("pca_5min.csv", datetime_format = "%Y-%m-%d %H:%M")

```

4.5 Visualizing results

Lets visualize the results, as a first look, and to check their reasonability. Both visualizations are useful to compare with the raw factors plotted in [src/data_factors.ipynb](#) / Chapter 2.

To start, lets visualize the resulting datasets.

One minute results:

```
data_pcs[1]
```

datetime datetime[s]	block u32	pc_1 f64	pc_2 f64	pc_3 f64	pc_4 f64	pc_5 f64	pc_6 f64
2001-01-02 09:30:00	0	-0.000154	0.000016	-0.00051	0.000063	0.000336	-0.000114
2001-01-02 09:31:00	0	0.000537	0.000525	-0.000502	-0.000011	0.000106	-0.000005
2001-01-02 09:32:00	0	0.000371	-0.00026	-0.000674	-0.000094	-0.000139	-0.000131
2001-01-02 09:33:00	0	0.001591	0.000637	0.000092	0.000128	-0.000198	-0.000318
2001-01-02 09:34:00	0	0.001138	0.000373	-0.000498	0.000055	-0.000262	-0.000116
...	...	...	...	...	...	...	...
2017-12-29 15:55:00	1919	0.000099	-0.000078	-0.000025	0.000016	0.000013	-0.000065
2017-12-29 15:56:00	1919	0.000108	-0.000052	-0.000007	-0.000003	0.000034	0.000017
2017-12-29 15:57:00	1919	0.000129	-0.000089	0.000145	-0.000099	-0.000007	0.000054
2017-12-29 15:58:00	1919	-0.000002	-0.000009	0.000012	0.000012	-0.000171	0.000012
2017-12-29 15:59:00	1919	0.000184	-0.001405	0.000664	-0.00039	0.000102	-0.000261

And five minute results:

```
data_pcs[5]
```

datetime datetime[s]	block u32	pc_1 f64	pc_2 f64	pc_3 f64	pc_4 f64	pc_5 f64	pc_6 f64
2001-01-02 09:30:00	0	0.003726	0.002074	-0.000139	0.000391	0.00043	0.000366
2001-01-02 09:35:00	0	-0.000655	0.003066	-0.002053	0.000727	0.000623	-0.000289
2001-01-02 09:40:00	0	-0.00405	0.000519	-0.002211	-0.000739	-0.001156	0.001771
2001-01-02 09:45:00	0	0.001775	0.000073	0.000385	-0.000416	-0.000813	0.001325
2001-01-02 09:50:00	0	0.001435	0.001594	-0.000542	-0.000514	-0.001209	0.000588

datetime	block	pc_1	pc_2	pc_3	pc_4	pc_5	pc_6
datetime[s]	u32	f64	f64	f64	f64	f64	f64
...	...	...	...	...	...	...	...
2017-12-29 15:35:00	857	-0.000216	0.000176	0.000693	0.000143	-0.000309	0.000039
2017-12-29 15:40:00	857	0.000002	0.000005	0.000292	0.000139	0.000093	-0.00013
2017-12-29 15:45:00	857	0.00021	0.000259	-0.000432	0.000171	0.000151	-0.000029
2017-12-29 15:50:00	857	0.000579	0.001401	0.000888	0.000196	-0.000146	-0.000361
2017-12-29 15:55:00	857	-0.000324	-0.000053	0.001916	0.000188	-0.00002	0.000153

Now, before plotting some graphs, lets store intermediate results for further analysis below.

```

n_fcts = len(cols_fct)
n_blocks = {k: res["evalues"].shape[0] for k, res in pcas.items()}

cols_evalue = {f"column_{i}": f"value_{i + 1}" for i in range(n_pcs)}
cols_evector = {f"column_{i}": f"vec_{i + 1}" for i in range(n_pcs)}

data_evals = {
    k: (
        pl.DataFrame(res["evalues"])
        .with_columns(block = pl.row_index())
        .rename(cols_evalue)
    )
    for k, res in pcas.items()
}

data_evecs = {
    k: (
        pl.DataFrame(res["evectors"].transpose(0, 1, 2).reshape((-1, n_fcts)))
        .rename(cols_evector)
        .with_columns(
            fct = pl.repeat(cols_fct, n_blocks[k]).explode(),
            block = pl.arange(0, n_blocks[k]).repeat_by(n_fcts).list.explode()
        )
    )
    for k, res in pcas.items()
}

```

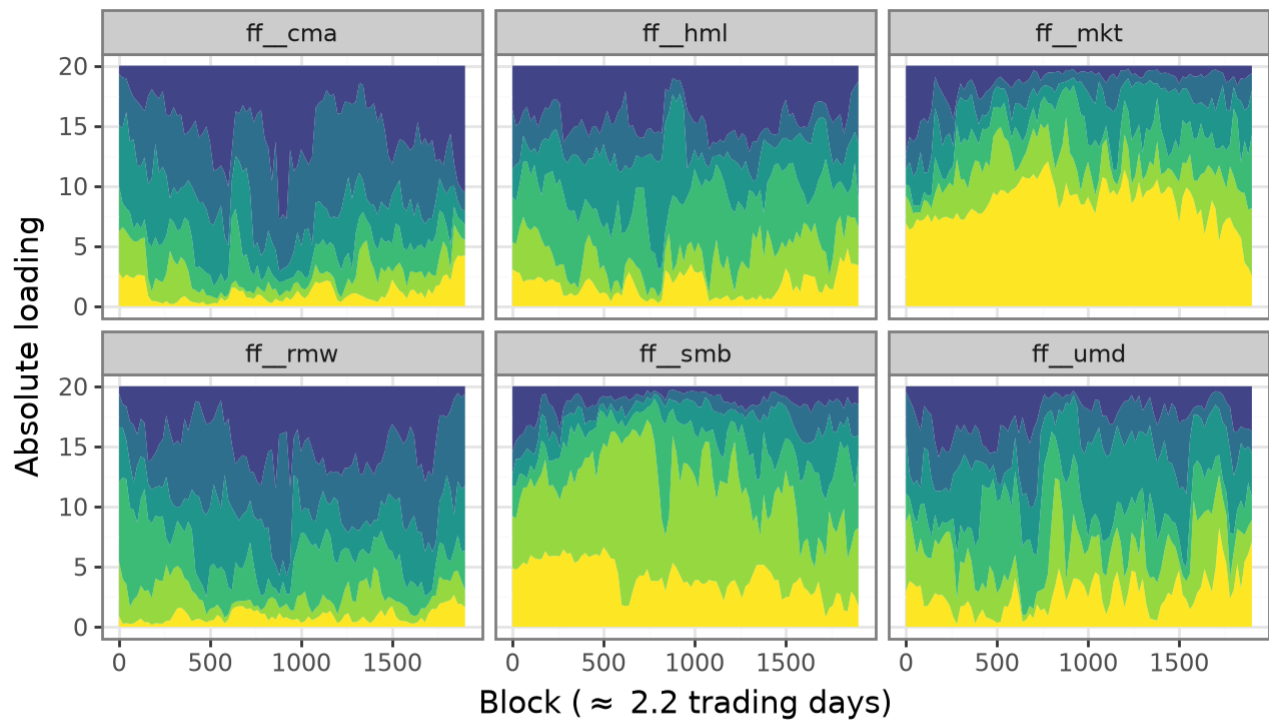
The factor loadings for each factor, for all PCs, across time:

```

plot.factor_loadings(
    data_evecs[1],
    list(cols_evector.values()),
    blocks_size[1] / PARS.blocks_1min,
    exp = 1
)

```

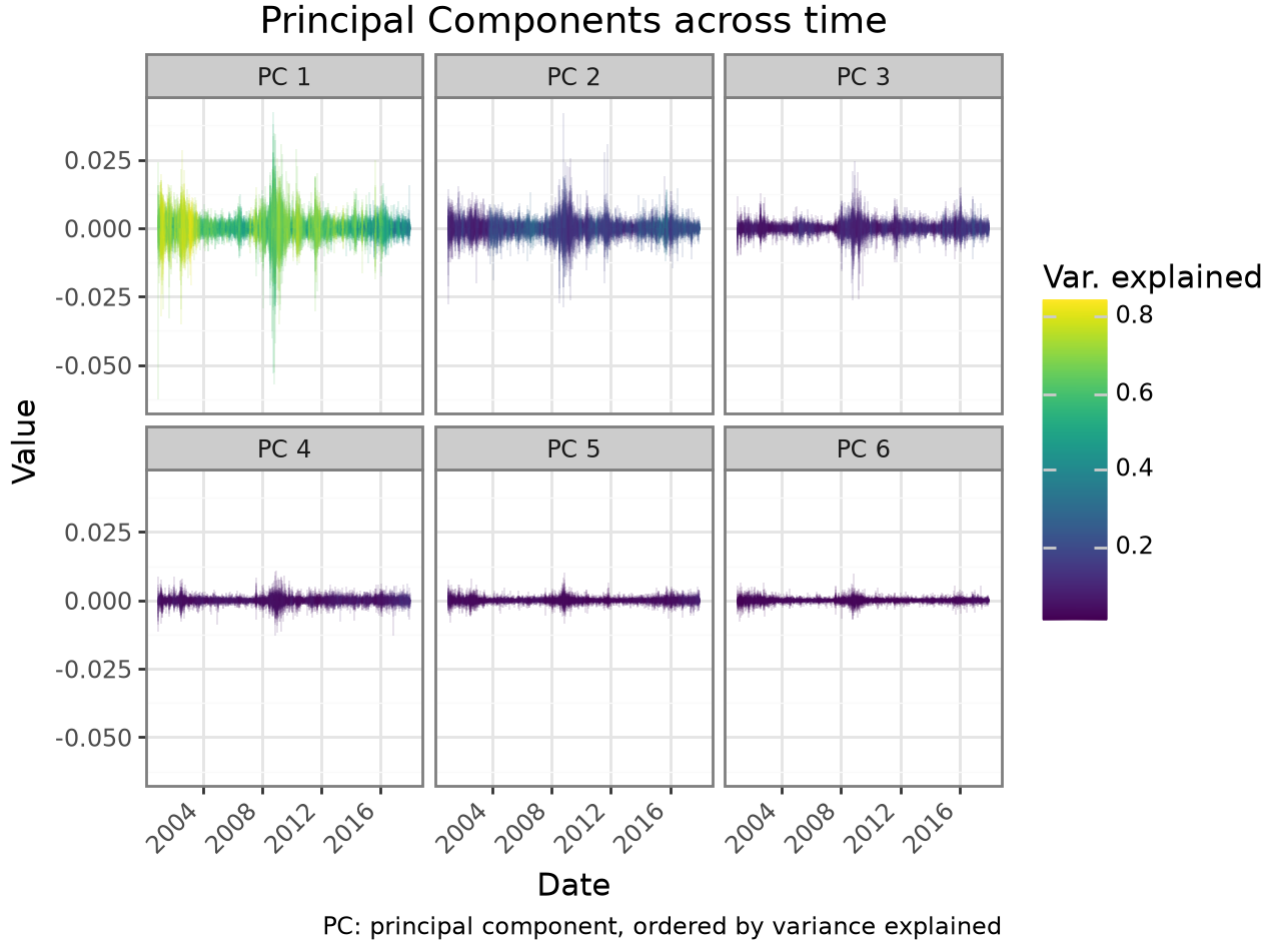
Factors' absolute loadings on PCs across time



PC: principal component, ordered by variance explained

Second, the principal components themselves, colored by the variance they explained at each block.

```
plot.principal_components(
    data_evals[1], data_pcs[1],
    blocks_size[1], list(cols_evalue.values()), list(cols_pc.values())
)
```



4.6 Appendix

4.6.1 Inference and spectral functions

The estimators will rely on the inference for $\int_0^t \lambda(c_s) ds$. When eigenvalues can be repeated, this is non-differentiable, and the authors introduce a ‘trick’ with spectral functions. This allows them to prove consistency and find bias-correction terms. The bias arises from the fact that the eigenvalues are non-linear functions of the covariance matrix. In reality, our measure of such matrix comes from the noisy high-frequency data, noise which when applied to non-linear operations, creates a bias.

In short, A spectral function $F(c_s)$ is simply a matrix function that is invariant to rotations. Associated with it is a symmetric function $(f \circ \lambda)(c_s)$ that takes the vector of eigenvalues. Studying the asymptotic of $\int_0^t F(c_s) ds$ allows the authors to prove consistency of their proposed estimators and add bias correction where needed. See the paper for more details, here, only the estimators are shown.

The authors make the four main assumptions below, although not all are needed for all results and their versions. They should be read in the context of the paper.

1. **Semimartingale Dynamics (Assumption 1):** The drift term b_t is progressively measurable and locally bounded, and the spot covariance matrix c_t is itself an Itô semimartingale. This standard assumption, along with limits on jump activity, ensures the underlying processes are mathematically well-behaved.
2. **Spectral Regularity for Consistency (Assumption 2):** The symmetric function f (where $F = f \circ \lambda$) is continuous and satisfies a polynomial growth condition $\|f(x)\| \leq K(1 + \|x\|^\zeta)$.

This guarantees that the target integral $\int_0^t F(c_s)ds$ is strictly well-defined, making the proposed estimators consistent.

3. **Non-Crossing Eigenvalues for Inference (Assumption 3):** To establish asymptotic distributions (like standard errors via the Central Limit Theorem), different groups of eigenvalues must not cross over within the time window $[0, t]$ (i.e., $c_s \in M^*(g_1, \dots, g_r)$). This prevents non-differentiable “kinks” from forming, ensuring the eigenvalue function remains sufficiently smooth (C^3) so the mean-value theorem can be applied.
4. **Weakened Non-Crossing (Assumption 4):** If we only care about a specific simple eigenvalue (e.g., isolating just the 1st Principal Component), Assumption 3 can be heavily relaxed. We only need to ensure that this specific target eigenvalue does not cross its immediate neighbor (e.g., $\lambda_1(c_s) > \lambda_2(c_s)$) during $[0, t]$. The crossover behavior of the remaining eigenvalues does not matter.

4.6.2 Integrated eigenvalues and eigenvectors

Integrated vector of eigenvalues $\int_0^t \lambda_s ds$: the estimator is a sum of the instantaneous eigenvalues, minus the bias correction term.

$$\hat{\lambda}_g = \underbrace{k_n \Delta_n \sum_i \hat{\lambda}_{g, ik_n \Delta_n}}_{\text{raw Estimator}} - \underbrace{\Delta_n \text{Tr} \left(\left(\hat{\gamma}_{g, ik_n \Delta_n} - \hat{c}_{ik_n \Delta_n} \right)^+ \hat{c}_{ik_n \Delta_n} \right) \hat{\gamma}_{g, ik_n \Delta_n}}_{\text{bias Correction}} \quad (17)$$

Integrated eigenvector $\int_0^t \gamma_{g,s} ds$: the estimator is a sum of the instantaneous eigenvectors, minus the bias correction term.

$$\hat{\gamma}_g = \underbrace{k_n \Delta_n \sum_{i=0}^{\lfloor t/(k_n \Delta_n) \rfloor} \hat{\gamma}_{i, k_n \Delta_n}}_{\text{raw estimator}} - \underbrace{\frac{1}{2k_n} \sum_{p \neq q} \frac{\hat{\lambda}_{g, ik_n \Delta_n} \hat{\lambda}_{p, ik_n \Delta_n} \hat{\lambda}_{g, ik_n \Delta_n}}{(\hat{\lambda}_{g, ik_n \Delta_n} - \hat{\lambda}_{p, ik_n \Delta_n})^2} \hat{\gamma}_{g, ik_n \Delta_n}}_{\text{bias correction}}$$

5 Realized betas and risk-adjusted returns

To be created...

6 Conditional alphas

To be created...

7 References

- Aït-Sahalia, Yacine, and Dacheng Xiu. 2015. “Principal Component Analysis of High Frequency Data.” Working Paper 21584. Working Paper Series. National Bureau of Economic Research. <https://doi.org/https://doi.org/10.3386/w21584>.
- Aleti, Saketh. 2022. “The High-Frequency Factor Zoo.” <https://doi.org/http://dx.doi.org/10.2139/ssrn.4021620>.